UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA

Ingeniería en ciencias y sistemas

Informe de desarrollo del proyecto 2

Laboratorio de Introducción a la Programación y Computación 1

Auxiliar: José Leonel López Ajvix

Estudiante: Oliver Jorge Raxtún Morales

Carné: 202400634

Introducción

El presente informe documenta el proceso de diseño, desarrollo e implementación del Proyecto 2 del curso Introducción a la Programación y Computación 1 de la Universidad San Carlos de Guatemala, correspondiente al primer semestre de 2026.

El proyecto consiste en el desarrollo de una aplicación de escritorio denominada Sancarlista Academy, un sistema de gestión académica que permite administrar instructores, estudiantes, cursos, inscripciones y calificaciones. La aplicación fue desarrollada utilizando el lenguaje de programación Java con la biblioteca gráfica Swing para la interfaz de usuario.

A lo largo del informe se detallan las decisiones de diseño tomadas, los problemas encontrados durante el desarrollo y las soluciones implementadas, así como una descripción del funcionamiento de cada módulo del sistema.

Descripción General del Sistema

Objetivo del Sistema

Sancarlista Academy es una plataforma académica de escritorio que permite gestionar de forma integral los procesos académicos de una institución educativa. El sistema contempla tres tipos de usuarios con diferentes niveles de acceso y funcionalidades:

- Administrador: Tiene control total sobre el sistema. Puede crear, editar y eliminar instructores, estudiantes y cursos. También gestiona inscripciones y visualiza la bitácora.
- Instructor: Puede gestionar las notas de los estudiantes en las secciones que le han sido asignadas por el administrador. Tiene acceso a la generación de reportes académicos.
- Estudiante: Puede inscribirse en secciones disponibles, consultar sus calificaciones y actualizar su información de perfil.

Tecnologías Utilizadas

Tecnología	Uso en el proyecto
Java SE 8+	Lenguaje principal de desarrollo
Java Swing	Construcción de la interfaz gráfica (GUI)
Serialización Java	Persistencia de datos mediante archivos .ser
Java Threads	Procesamiento concurrente en segundo plano
NetBeans IDE	Entorno de desarrollo integrado (IDE)
Arreglos nativos []	Estructuras de datos (sin ArrayList ni HashMap)

Patrones de Diseño Aplicados

El sistema implementa el patrón Modelo-Vista-Controlador (MVC) como arquitectura principal:

- Modelo: Paquete modelo/ — Contiene las entidades Usuario, Instructor, Estudiante, Administrador, Curso, Nota, EntradaBitacora y SistemaDatos. Aplica herencia con la clase abstracta Usuario como base.
- Vista: Paquete vista/ — Contiene todas las interfaces gráficas Swing: LoginFrame, AdminFrame, InstructorFrame, EstudianteFrame y ReportesFrame.
- Controlador: Paquete controlador/ — La clase SistemaController centraliza toda la lógica de negocio y coordina las operaciones entre Vista y Modelo.

Implementación del Proyecto

Módulo de Autenticación

El módulo de autenticación fue implementado en la clase LoginFrame. Al iniciar el sistema, el usuario debe ingresar su código y contraseña. El controlador busca el usuario en SistemaDatos y valida las credenciales.

Un aspecto importante de la implementación es el usuario administrador precargado. Este se inicializa en el constructor de SistemaDatos con código 'admin' y contraseña compuesta por 'IPC1' más la letra de la sección. Este usuario se crea únicamente cuando no existe un archivo de serialización previo.

La redirección por roles se implementó mediante un switch que, según el valor del atributo rol del objeto Usuario autenticado, abre el Frame correspondiente (AdminFrame, InstructorFrame o EstudianteFrame) y cierra el LoginFrame.

Jerarquía de Clases (POO)

Uno de los pilares del proyecto fue la aplicación correcta de los principios de Programación Orientada a Objetos. La jerarquía de clases se diseñó de la siguiente manera:

- La clase abstracta Usuario contiene los atributos comunes a todos los usuarios (codigo, nombre, fechaNacimiento, genero, contrasena, rol) y declara el método abstracto getDescripcion(), que obliga a cada subclase a implementar su propia descripción.
- Administrador hereda de Usuario sin agregar atributos adicionales, ya que su comportamiento especial está determinado por su rol.
- Instructor hereda de Usuario y agrega un arreglo de Strings para almacenar los códigos de secciones asignadas, con métodos como tieneSeccion(), agregarSeccion() y eliminarSeccion().
- Estudiante hereda de Usuario y agrega un arreglo para las secciones inscritas, con métodos como estaInscrito(), inscribirSeccion() y desasignarSeccion().

Esta implementación aplica el polimorfismo, ya que el controlador puede tratar a todos los usuarios como objetos de tipo Usuario y utilizar instanceof para acceder a las funcionalidades específicas de cada rol.

Almacén de Datos (SistemaDatos)

Dado que el uso de colecciones como ArrayList, HashMap o LinkedList está prohibido por las especificaciones del proyecto, se implementó la clase SistemaDatos usando arreglos nativos de Java con capacidades predefinidas:

- usuarios: arreglo de 500 posiciones para todos los tipos de usuario
- cursos: arreglo de 200 posiciones
- notas: arreglo de 5,000 posiciones
- bitacora: arreglo de 10,000 posiciones para entradas de bitácora

Las operaciones de eliminación se implementaron con el algoritmo de desplazamiento: al eliminar un elemento en la posición i, todos los elementos desde i+1 hasta el final se desplazan una posición hacia adelante, y el último elemento válido se establece en null. Esto mantiene la integridad del arreglo sin 'huecos'.

Persistencia con Serialización

La persistencia de datos se implementó mediante la serialización nativa de Java. La clase SistemaDatos implementa la interfaz Serializable, lo que permite convertir el objeto completo (incluyendo todos sus arreglos de datos) en un flujo de bytes que se almacena en el archivo datos_sistema.ser.

La clase utilitaria Serializacion.java encapsula este proceso con dos métodos: guardar() que usa ObjectOutputStream para escribir el objeto al disco, y cargar() que usa ObjectInputStream para leer el archivo existente o crear un nuevo SistemaDatos si el archivo no existe o está corrupto.

El método guardar() se invoca automáticamente al final de cada operación que modifica datos (crear, actualizar, eliminar), garantizando que los cambios persistan inmediatamente.

Procesamiento con Hilos

El sistema implementa tres hilos de fondo usando la interfaz Thread de Java. Cada hilo se ejecuta en un bucle while controlado por una bandera booleana volatile llamada hilosActivos. El uso de volatile garantiza la visibilidad del valor actualizado entre hilos.

Los hilos implementados son:

- Thread-Sesiones (cada 10 seg): Muestra el conteo de usuarios activos y el timestamp de última actividad en la barra de estado de AdminFrame.
- Thread-Inscripciones (cada 8 seg): Simula un contador de inscripciones pendientes en la barra de estado de EstudianteFrame.
- Thread-Estadísticas (cada 15 seg): Muestra totales de cursos, estudiantes y notas en InstructorFrame.

La actualización de la interfaz gráfica desde hilos secundarios se realiza correctamente usando SwingUtilities.invokeLater(), que encola la actualización en el Event Dispatch Thread (EDT) de Swing para evitar condiciones de carrera en la GUI.

Al cerrar sesión, se establece hilosActivos = false, lo que hace que el bucle while termine en la siguiente iteración. Los hilos son daemon (setDaemon(true)), lo que significa que la JVM los termina automáticamente si no quedan hilos no-daemon activos.

Módulo de Notas y Cálculo de Promedio

El registro de notas incluye múltiples validaciones en cadena implementadas en el método crearNota() del controlador:

1. Verificar que el curso existe en SistemaDatos
2. Verificar que el instructor tiene asignada la sección especificada
3. Verificar que el estudiante está inscrito en esa sección

4. Verificar que la nota esté en el rango 0-100
5. Verificar que la ponderación sea mayor que 0

El promedio ponderado se calcula con la fórmula: Promedio = Σ(Nota × Ponderación) / Σ(Ponderación). Esta implementación permite que las ponderaciones no necesariamente sumen 100%, calculando el promedio relativo correctamente. Un estudiante se considera aprobado si su promedio es mayor o igual a 61.

Bitácora del Sistema

La bitácora registra automáticamente cada evento significativo del sistema. Se implementó la clase utilitaria Bitacora.java con el método estático registrar(), que recibe el tipo de usuario, código de usuario, nombre de la operación, estado (EXITOSA/FALLIDA) y descripción detallada.

Este método se invoca al final de cada operación en el controlador, tanto si fue exitosa como si falló, permitiendo un registro completo de auditoría. La bitácora puede visualizarse desde la pestaña correspondiente en AdminFrame y exportarse a CSV.

Problemas Encontrados y Soluciones Adoptadas

#	Problema Encontrado	Solución Implementada	Lección Aprendida
1	La contraseña del administrador no se actualizaba al cambiarla en el código fuente.	El archivo datos_sistema.ser cargado tiene prioridad sobre el constructor. Solución: eliminar el .ser para forzar recreación del sistema de datos.	La serialización persiste el objeto original; cambios en código no afectan archivos ya serializados.
2	Actualización de la interfaz gráfica desde hilos secundarios causaba excepciones en Swing.	Todas las actualizaciones de UI desde hilos se envuelven en SwingUtilities.invokeLater() para ejecutarse en el EDT.	Swing no es thread-safe; cualquier modificación de componentes debe hacerse desde el EDT.
3	Al eliminar elementos del arreglo quedaban posiciones null intercaladas, causando errores al iterar.	Se implementó el algoritmo de desplazamiento: al eliminar, los elementos posteriores se mueven una posición hacia adelante.	Sin colecciones dinámicas, la gestión manual del arreglo es crítica para mantener la integridad.
4	Los hilos seguían ejecutándose después de cerrar sesión, causando actualizaciones en la UI cerrada.	Se usa una bandera booleana volatile hilosActivos que se establece en false al cerrar sesión, terminando el bucle del hilo.	El manejo del ciclo de vida de los hilos es esencial para evitar memory leaks y errores de UI.
5	La carga de archivos CSV con caracteres especiales (tildes, ñ) fallaba o mostraba caracteres incorrectos.	Se usa BufferedReader con el charset del sistema por defecto. Se recomienda que los CSV usen UTF-8.	El manejo de codificación de caracteres es importante en aplicaciones con texto en español.
6	Al seleccionar una fila en una tabla Swing y luego hacer operaciones, la tabla no se actualizaba automáticamente.	Se implementó un objeto Runnable (cargarTabla) que se invoca explícitamente después de cada operación de escritura para refrescar el DefaultTableModel.	DefaultTableModel no se auto-actualiza; es necesario llamar setRowCount(0) y repoblar la tabla.
7	El cálculo de promedio devolvía NaN cuando no había notas registradas (división por cero).	Se agregó una verificación: si Σ(Ponderación) == 0, retornar 0.0 directamente sin hacer la división.	Siempre validar denominadores antes de divisiones en cálculos de promedio.

Capturas de Pantalla del Sistema

A continuación se presentan las principales pantallas del sistema Sancarlista Academy en funcionamiento:

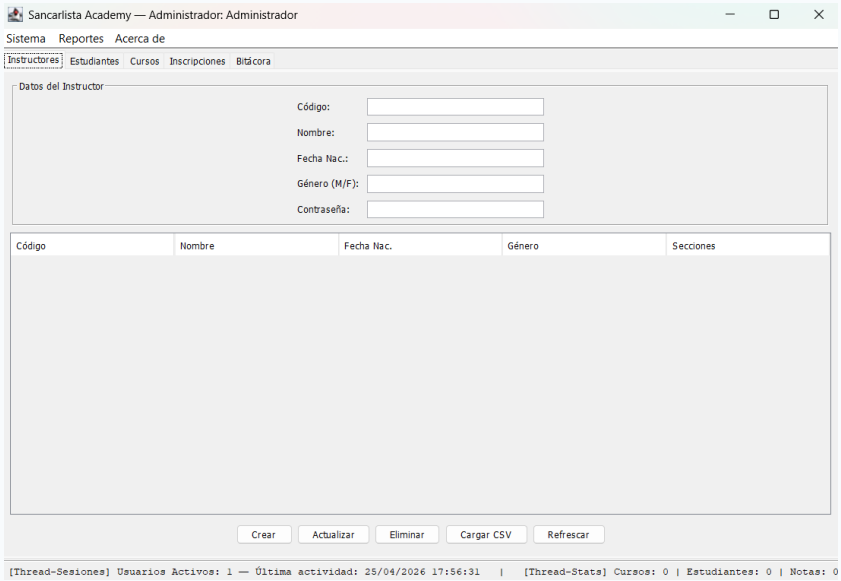
Pantalla de Login

Login del Sistema

Pantalla inicial con campos de código y contraseña. Validación de credenciales y redirección por rol.

Panel del Administrador — Gestión de Instructores

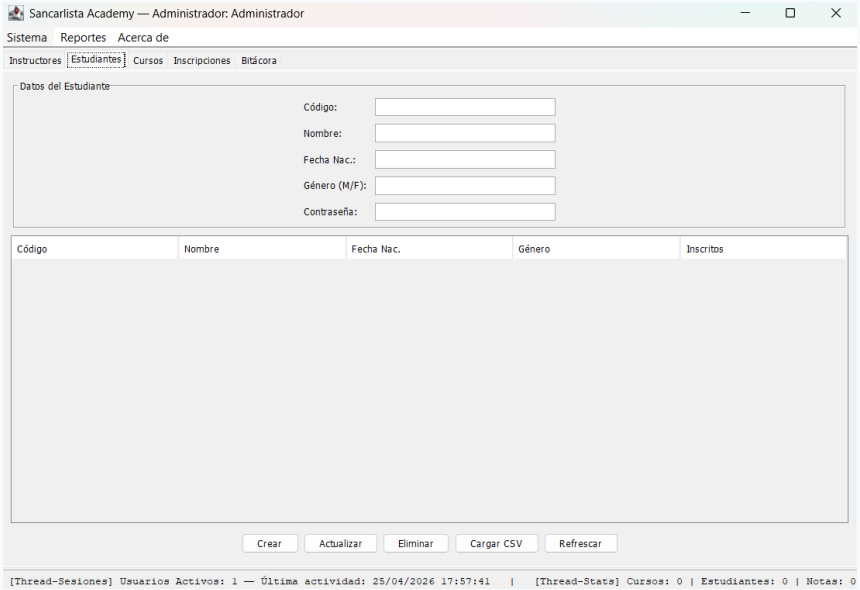
AdminFrame — Pestaña Instructores



Formulario de creación/edición de instructores y tabla con todos los registros del sistema.

Panel del Administrador — Gestión de Estudiantes

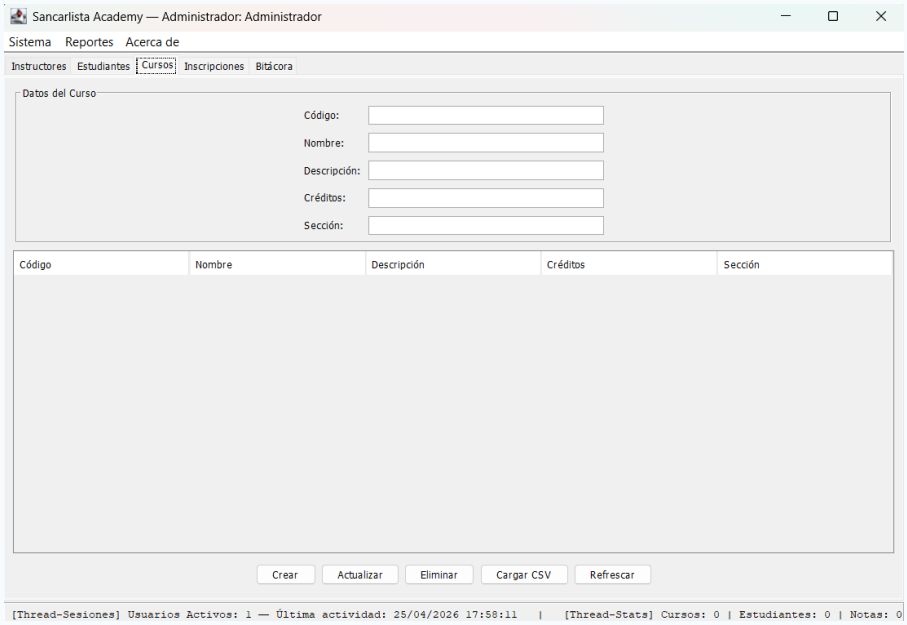
AdminFrame — Pestaña Estudiantes



CRUD completo de estudiantes con soporte de carga masiva desde archivos CSV.

Panel del Administrador — Gestión de Cursos

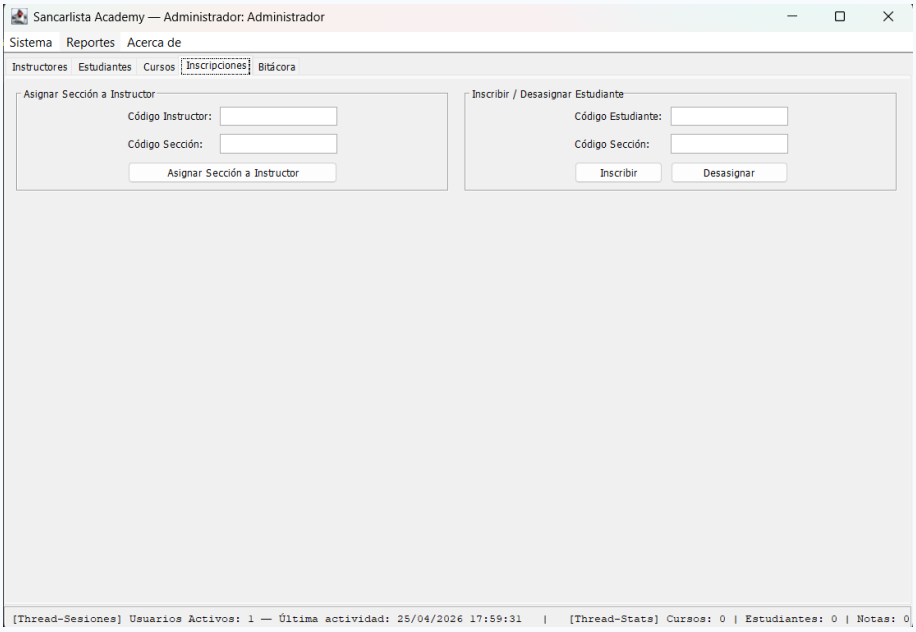
AdminFrame — Pestaña Cursos



Catálogo de cursos disponibles con campos de código, nombre, descripción, créditos y sección.

Panel del Administrador — Inscripciones

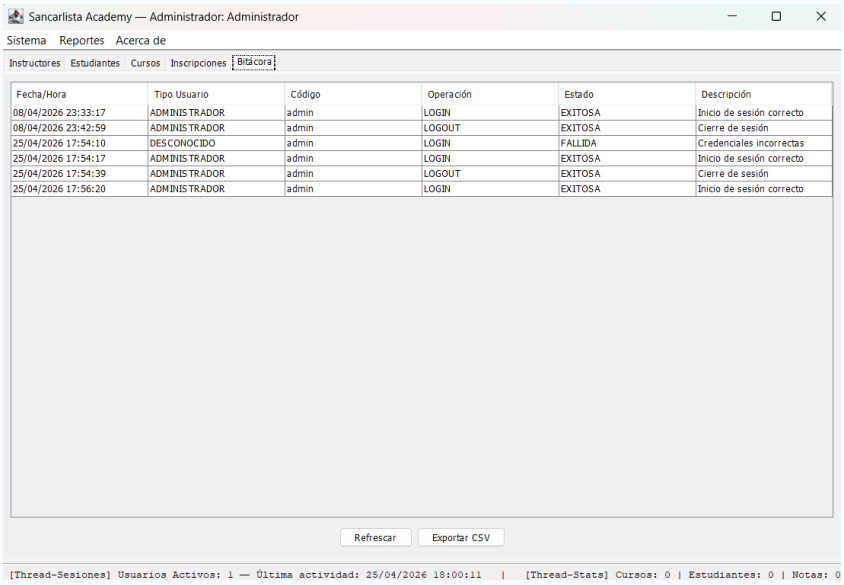
AdminFrame — Pestaña Inscripciones



Asignación de secciones a instructores e inscripción/desasignación de estudiantes.

Panel del Administrador — Bitácora

AdminFrame — Pestaña Bitácora



Fecha/Hora	Tipo Usuario	Código	Operación	Estado	Descripción
08/04/2026 23:33:17	ADMINISTRADOR	admin	LOGIN	EXITOSA	Inicio de sesión correcto
08/04/2026 23:42:59	ADMINISTRADOR	admin	LOGOUT	EXITOSA	Cierre de sesión
25/04/2026 17:54:10	DESCONOCIDO	admin	LOGIN	FALLIDA	Credenciales incorrectas
25/04/2026 17:54:17	ADMINISTRADOR	admin	LOGIN	EXITOSA	Inicio de sesión correcto
25/04/2026 17:54:39	ADMINISTRADOR	admin	LOGOUT	EXITOSA	Cierre de sesión
25/04/2026 17:56:20	ADMINISTRADOR	admin	LOGIN	EXITOSA	Inicio de sesión correcto

Registro completo de todos los eventos del sistema con opción de exportar a CSV.

Panel del Instructor — Gestión de Notas

InstructorFrame — Gestión de Notas

Formulario de registro de notas con validaciones de sección asignada y estudiante inscrito.

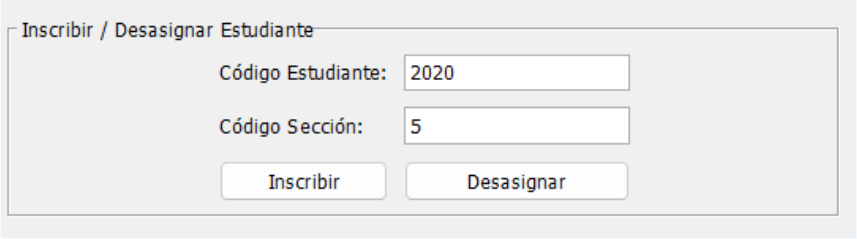
Panel del Estudiante — Mis Cursos

EstudianteFrame — Mis Cursos

Resumen de secciones inscritas con promedio ponderado y estado Aprobado/Reprobado.

Panel del Estudiante — Inscripción

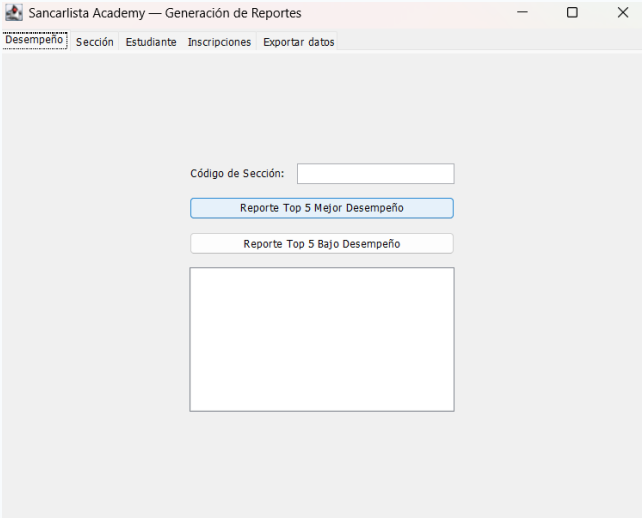
EstudianteFrame — Inscribir Sección



Catálogo de cursos disponibles para inscripción con validación de duplicados.

Módulo de Reportes

ReportesFrame — Generación de Reportes



Ventana de reportes con pestañas para desempeño, sección, estudiante e inscripciones.

Barra de Estado — Hilos en Tiempo Real

Barra de estado — Hilos activos

```
[Thread-Sesiones] Usuarios Activos: 1 — Última actividad: 25/04/2026 18:09:21   |   [Thread-Stats] Cursos: 0 | Estudiantes: 1 | Notas: 0
```

Barra inferior mostrando sesiones activas, estadísticas y timestamp actualizado por los Threads.

Estructura del Repositorio GitHub

El repositorio fue organizado siguiendo las especificaciones del enunciado del proyecto:

```
IPC1<Seccion>Proyecto2_<Carnet>_1S26/
│
├── Proyecto2/                    ← Código fuente Java
│   └── src/
│       ├── Main.java
│       ├── modelo/
│       ├── vista/
│       ├── controlador/
│       └── util/
│
├── README.md                     ← Descripción del proyecto
├── ManualTecnico.pdf             ← Manual técnico
├── ManualUsuario.pdf             ← Manual de usuario
├── DiagramaClases.pdf            ← Diagrama de clases UML
├── DiagramaFlujo.pdf             ← Diagramas de flujo
├── InformeDesarrollo.pdf         ← Este informe
└── dist/
    └── SancarlisitaAcademy.jar   ← Ejecutable empaquetado
```

Instrucciones de Compilación y Ejecución

Desde NetBeans (recomendado)

6. Abrir NetBeans IDE
7. File → Open Project → seleccionar la carpeta Proyecto2
8. Verificar que Main.java esté configurado como clase principal
9. Presionar F6 (Run Project) o el botón verde de ejecución
10. Para generar el JAR: click derecho en el proyecto → Clean and Build → el JAR aparece en dist/

Desde línea de comandos

```
# 1. Compilar todos los archivos
javac -d bin -sourcepath src src/Main.java

# 2. Ejecutar desde clases compiladas
java -cp bin Main

# 3. Crear JAR ejecutable
jar cfe SancarlisitaAcademy.jar Main -C bin .

# 4. Ejecutar el JAR
java -jar SancarlisitaAcademy.jar
```

Credenciales de acceso

Rol	Código	Contraseña
Administrador	admin	IPC1F (o tu sección)
Instructor	Creado por Admin	Definida al crear
Estudiante	Creado por Admin	Definida al crear

Conclusiones

Sobre la aplicación de POO

El desarrollo del proyecto permitió aplicar de forma práctica y concreta los principios de la Programación Orientada a Objetos. La jerarquía de herencia con la clase abstracta Usuario demostró cómo es posible compartir comportamiento común entre clases relacionadas, mientras que el polimorfismo permite tratar objetos de diferentes tipos de forma uniforme. El encapsulamiento fue fundamental para proteger la integridad de los datos del sistema.

Sobre el patrón MVC

La implementación del patrón Modelo-Vista-Controlador demostró sus ventajas en términos de organización y mantenibilidad del código. La separación clara de responsabilidades permite modificar la interfaz gráfica sin afectar la lógica de negocio, y viceversa. Esto facilita el mantenimiento y la extensión del sistema en el futuro.

Sobre la persistencia y serialización

La serialización de objetos Java resultó ser una forma efectiva y sencilla de implementar persistencia sin depender de bases de datos externas. Sin embargo, se aprendió que las modificaciones al código fuente no afectan automáticamente los datos ya serializados, lo que requiere gestión cuidadosa del archivo .ser durante el desarrollo.

Sobre el manejo de hilos

La implementación de hilos de fondo demostró cómo es posible ejecutar procesos concurrentes sin bloquear la interfaz de usuario. El uso de SwingUtilities.invokeLater() y las banderas volatile fueron fundamentales para garantizar la seguridad en el acceso a recursos compartidos entre hilos.

Sobre las restricciones del proyecto

Trabajar sin colecciones como ArrayList o HashMap requirió implementar manualmente operaciones que normalmente están disponibles en la API de Java. Esto resultó en una comprensión más profunda de cómo funcionan internamente estas estructuras de datos, especialmente en cuanto a la gestión de memoria y el desplazamiento de elementos al eliminar.

Referencias Bibliográficas

- Deitel, P. & Deitel, H. (2016). Cómo programar Java, 10ª edición. Pearson Education.
- Joyanes, L. (s.f.). Java 2: Manual de Programación. McGraw-Hill.
- Oracle Corporation. (2023). Java SE Documentation. https://docs.oracle.com/en/java/
- Oracle Corporation. (2023). Creating a GUI With Swing. https://docs.oracle.com/javase/tutorial/uiswing/
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.